Snake Arcade

Proyecto de Laboratorio de Programación III • por Najmah Zablah

Introducción

¿Qué es Snake Arcade?

Snake Arcade es un videojuego implementado en C++ con SFML 2.6.2, desarrollado como proyecto final de Laboratorio de Programación III (LCP208). Combina lógica orientada a objetos, manejo de archivos, algoritmos de ordenamiento y una interfaz gráfica no bloqueante en tiempo real.

- Structs & Clases
- Punteros
- fstream (archivos)
- Bubble Sort

C++17 • SFML 2.6.2 • Qt Creator • MinGW 64-bit

Arquitectura General

Separación en capas

main.cpp — Punto de entrada · Game loop

Renderer — Coordinador · Delega por estado

Pantallas ×4 — Menu · Selección · Juego · Scores

Juego — Lógica · Colisiones · Usuarios · HS

Serpiente — Movimiento · Cuerpo · Colisión propia

Lógica del Juego

Game.h / Game.cpp · class Juego

EstadoJuego

9 estados: MENU · CREAR_CUENTA · SELECCION · NIVELES · JUGANDO · PAUSANDO · GAME_OVER · HIGH_SCORES

Nivel

UNO=1 / DOS=2 / TRES=3 (valores enteros para serialización directa)

iniciar(nivel)

Reinicia serpiente, puntaje, obstáculos y velocidad. Genera los primeros coleccionables.

actualizar()

Llama a serpiente.mover() y verificarColisiones() solo en estado JUGANDO.

verificarColisiones()

Obtiene Coordenada* cabeza. Verifica wrap-around, muros, auto-colisión, obstáculos, aceitunas y estampilla.

Bubble Sort

ordenarPuntajes() ordena de mayor a menor. Máx. 10 registros por archivo.

Class Serpiente

Zerpent.h / Zerpent.cpp

Representa la serpiente internamente como un std::vector<Coordenada> donde el índice 0 es siempre la cabeza.

`getCabeza()`	Retorna Coordenada* → puntero a cuerpo[0]
`mover()`	insert al frente + pop_back (si !creciendo)
`cambiarDireccion()`	Ignora dirección opuesta
`crecer()`	Activa flag creciendo = true
`colisionConSiMisma()`	Compara cabeza vs cuerpo[1..n]

Estructuras de datos

Coordenada

`int x, y + operator==`

Aceituna

`posicion · tipo · puntos · activa`

Estampilla

`posicion · id · recolectada · visible · puntosBono`

PuntajeRecord

`nombreJugador · puntaje · nivel`

Niveles de Juego

Dificultad progresiva

01

Fácil

0.15 s/tick

Velocidad constante · Sin obstáculos internos · Wrap-around: la serpiente atraviesa los muros y reaparece en el extremo opuesto.

02

Medio

0.12 s → acelerado

Muros perimetrales letales · Velocidad incremental: factor = 1 − (puntaje/50)×0.05, mínimo ×0.4.

03

Difícil

0.10 s → acelerado

8–12 obstáculos aleatorios · Aceitunas doradas (30% probabilidad, +25 pts, velocidad ×0.7 temporal).

Capa Visual

Renderer + 4 clases Pantalla*

PantallaMenu

Login · Registro · Confirmar eliminación. Mantiene estado de formulario (inputNombre, inputContrasena, mensajeError).

PantallaSeleccion

Menú de usuario · Selección de nivel · Overlay 'Ver Estampillas'.

PantallaJuego

Tablero · Serpiente con ojos y sombrero · Glow de coleccionables · Overlays de pausa, game over y estampilla.

PantallaScores

Top 10 por nivel con tabs. Colores dorado / plata / bronce para top 3. Carga puntajes ya ordenados.

Manejo de Archivos

fstream · Persistencia de datos

usuarios.txt

```
nombre_usuario|contraseña
Najmah|abc123
```

Escritura: ofstream — sobrescribe completamente.
Lectura: ifstream + getline(), split por '|'.

highscores.txt

```
nombreJugador|puntaje|nivel
Najmah|210|1
```

1 record por jugador por nivel. Actualiza solo si puntaje > anterior. Siempre top 10.

Verificación contra npos antes de substr() — evita excepciones en archivos malformados.

Detalles Visuales

Serpiente · Efectos · Estampillas

Ojos dinámicos

Se reposicionan según dirección (dx = cuerpo[0].x − cuerpo[1].x, normalizado). 3 capas: blanco, pupila y reflejo.

Sombrero de copa

Ala + cuerpo negro + cinta morada (RGB 75,55,110). Dibujado sobre la celda de la cabeza con 4 RectangleShape.

Glow por capas

N círculos concéntricos con alfa decreciente alrededor de aceitunas (verde/dorado) y estampillas (blanco).

Overlay estampilla

Fade-in/out de 2.8 s. Alpha interpolado: tramo 0–20% (entrada) y 85–100% (salida). Solo activo en JUGANDO.

01 SECTION

Requisitos

Académicos Cumplidos

Requisitos Cumplidos

LCP208 — Verificación

Structs	Coordenada · Aceituna · Estampilla · PuntajeRecord	✓
Clases	Serpiente · Juego · PantallaMenu · PantallaJuego · Renderer...	✓
Punteros	getCabeza() retorna Coordenada* → modifica cabeza in situ	✓
fstream	usuarios.txt y highscores.txt con ifstream/ofstream	✓
Bubble Sort	ordenarPuntajes() $O(n^2)$, $n \leq 10$, descendente	✓
GUI no bloqueante	pollEvent() + sf::Clock — sin detener el hilo principal	✓

¡Gracias!

Najmah Karyme Zablah • 22441365

3	9	10	4
Niveles	Archivos .cpp/.h	High Scores	Pantallas

C++17 • SFML 2.6.2 • Qt Creator • MinGW 64-bit